PragmaTech

INDUSTRY REPORT · NO. 01 / 2026

VOL. 01 2026

MAY 12–18, 2026 // 92 ENGINEERS // 3 CONTINENTS

The AI Productivity *Paradox.*

Why teams shipping 30× more code with AI still feel slower — and what **92 Spring Boot engineers** say is actually holding them back in 2026.

92	43%	21.9%	25%
COMPLETED RESPONSES	SURVEY CONVERSION	BIGGEST FRUSTRATION: SLOW BUILD & TEST FEEDBACK	BLAME AI-GENERATED CODE ITSELF

§ 01 / EXECUTIVE SUMMARY

The promise doesn't match the pipeline.

AI tools promised to make us 10× faster. The largest independent studies of 2025 and 2026 tell a more uncomfortable story — and our own data from 92 engineers confirms it.

The two studies framing this report

Google's **2025 DORA Report** — surveying nearly 5,000 technology professionals — found that **90% of teams now use AI** and **80% feel more productive**, yet AI adoption still has a *negative* relationship with software delivery stability. The report's central conclusion: AI doesn't fix a team; it amplifies whatever is already there.

The **METR randomized controlled trial** (2025) measured experienced open-source developers on real tasks. Result: they thought AI made them **20% faster**. It actually made them **19% slower**. The bottleneck was reviewing, debugging, and verifying generated code — precisely the work that gets harder on a flaky test pipeline.

What our 92 engineers say

The same paradox shows up in our data. **30%** name quality and reliability as their absolute highest priority — ahead of speed. But the things blocking them are not product complexity. They are **slow tests, flaky pipelines, unreviewed AI output, and legacy code drowning in technical debt**.

For Spring Boot teams, the picture sharpens: **19%** say their biggest testing bottleneck is simply *getting everyone on the same testing strategy*. Tools are not the constraint. **Alignment, integration tests, and feedback loops are.**

Four numbers to remember

21.9%
SAY LONG BUILD & TEST TIMES ARE THEIR #1 FRUSTRATION

25%
FRUSTRATED BY ANALYZING & DEBUGGING AI-GENERATED CODE

21.4%
BLOCKED BY UNMANAGEABLE LEGACY CODE

19.0%
SPRING BOOT TEAMS: NO SHARED TESTING STRATEGY

> We feel like we are generating 30× more code using AI. But how do we reliably secure this code without our test pipelines turning into a maintenance time bomb?
>
> — THE QUESTION BEHIND THIS STUDY

§ 02 / METHODOLOGY

How the numbers were gathered.

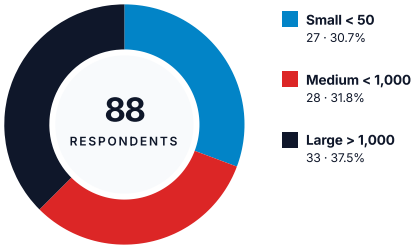
Six questions. Three minutes. One week of fieldwork across a self-selected audience of working Java and Spring Boot engineers.

Fieldwork

FIELD	VALUE
Period	12 – 18 May 2026
Survey visits	214
Completed responses	92
Completion rate	43.0%
Leads captured	60
Channels	Newsletter, LinkedIn
Questions	6 (4 open · 2 multi)

Company size of respondents

N = 88 · QUESTION 5



A note on the sample

The audience self-selected via a developer newsletter and LinkedIn distribution focused on Java and Spring Boot. The sample skews toward practitioners actively thinking about testing and AI — not the "not-using-AI-yet" long tail. Read the findings as a snapshot of *engaged, AI-aware* engineering teams rather than the global average.

Why this matters

The respondent base is split almost evenly across small, mid-market and enterprise. Findings hold whether you ship a 5-developer SaaS or operate a Spring Boot estate inside a Fortune 500.

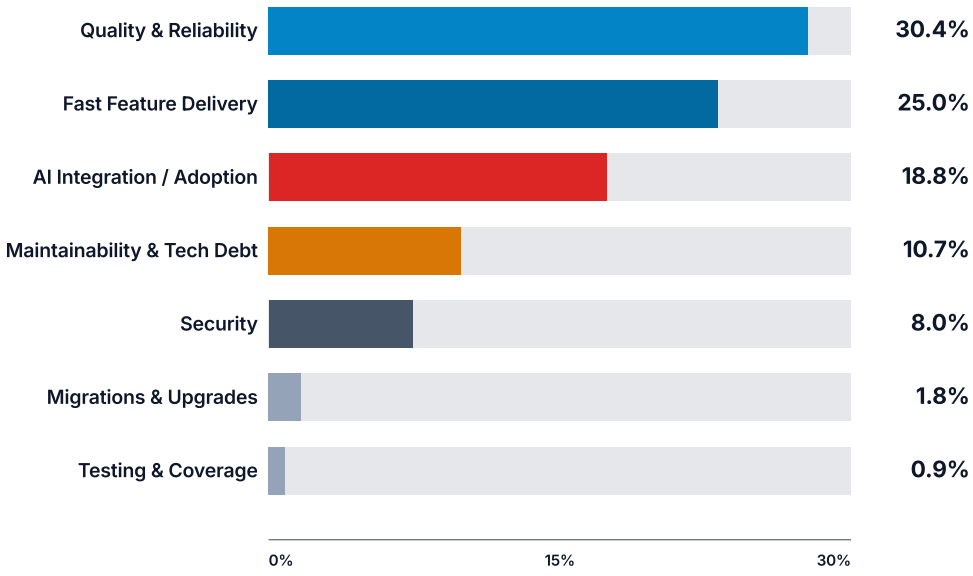
§ 03 / WHAT'S TOP OF MIND

Quality still wins.

We asked 112 engineers what their team's absolute highest priority is right now. Free-text answers, hand-coded into seven themes. *Some answers map to more than one.*

Q1 — "What is currently the absolute highest priority for you and your engineering team?"

N = 112 FREE-TEXT ANSWERS · MULTI-CODED · SHARE OF RESPONDENTS MENTIONING THEME



> Shipping features reliably with minimal or no technical debt, supported by tests that effectively identify regressions and expose bugs.
>
> — RESPONDENT #81

The signal

Quality and speed are not opposites in this dataset. Engineers want both — in the same answer. Almost a quarter of respondents explicitly paired them: "deliver fast *with* high quality", "faster delivery with higher quality", "ship features quicker while tackling technical debt."

AI is named directly by **18.8%** — but always with the same hedge: *"leverage AI, but in a controlled manner"*; *"use AI tools but review code"*; *"not reduce code quality while using Gen AI as much as possible".*

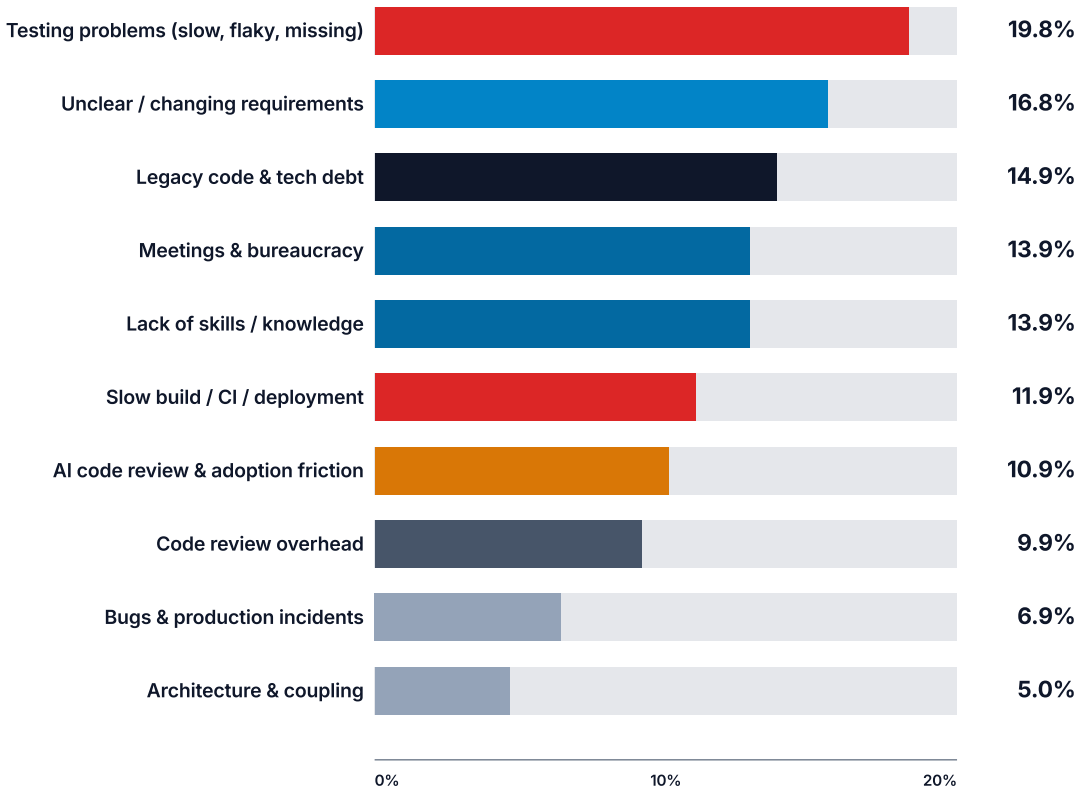
§ 04 / THE REAL BOTTLENECKS

What is actually blocking us.

101 free-text answers to “what is holding you back from shipping features quickly?”, coded into ten themes. *One in five teams blames their test suite.*

Q2 — What is currently holding you back the most from shipping new features quickly and smoothly?

N = 101 FREE-TEXT ANSWERS · MULTI-CODED · SHARE OF RESPONDENTS MENTIONING THEME



> Slow and painful integration tests, dead code and unused functionalities, lack of observability.
>
> RESPONDENT · #53

> Review time gets longer due to bigger and more complicated AI-generated merge requests.
>
> RESPONDENT · #12

> Interpreting the tests to grasp dependencies is challenging — sometimes they’re just AI-generated noise.
>
> RESPONDENT · #73

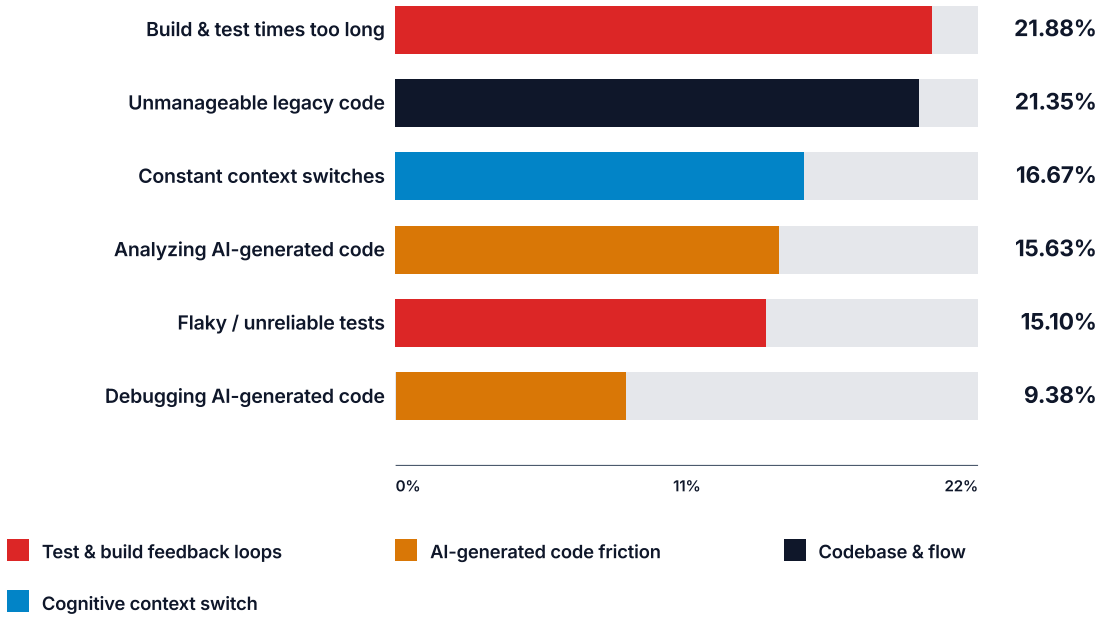
§ 05 / THE FRICTION MAP

Where the day goes.

Forced-choice question: pick what frustrates you most about your current deployment and testing process. Multiple selections allowed.

Q3 — What frustrates you the most about your current deployment or testing process?

192 SELECTIONS FROM 92 RESPONDENTS · % OF TOTAL SELECTIONS SHOWN



The hidden math of this chart

Grouped by theme, the picture changes. **Test & build pipeline friction** (slow times + flaky tests) accounts for **36.98%** of all selections. **AI-generated code friction** (analyzing + debugging) accounts for **25.01%**. The two together — a productivity tax on every AI-assisted change — eat **three out of five** frustration votes in the sample.

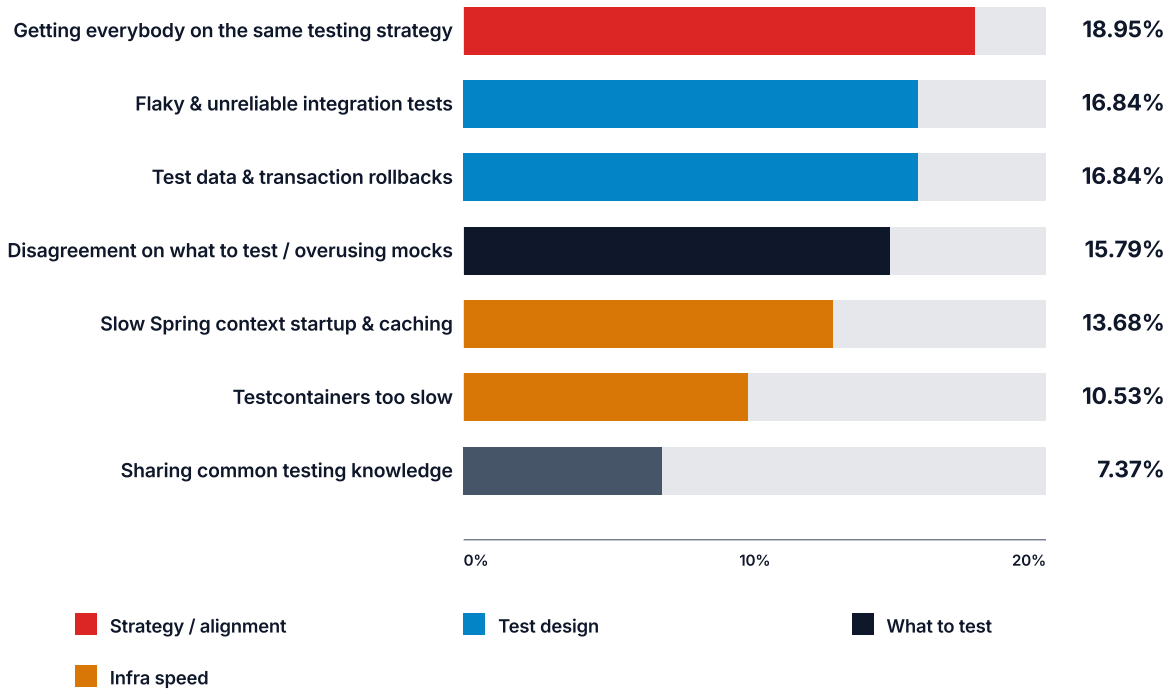
§ 06 / THE SPRING BOOT TESTING STACK

Where Spring Boot teams get stuck.

For automated testing inside Spring Boot codebases, what is your team's biggest bottleneck right now? *Pick your top frustration.*

Q6 — Spring Boot automated testing bottlenecks

N = 95 SELECTIONS · % OF TOTAL SELECTIONS SHOWN



The pattern hiding in the data

The top three answers — strategy alignment (18.95%), flaky integration tests (16.84%), and test data management (16.84%) — are **not tooling problems**. Spring Boot, JUnit 5 and Testcontainers all *can* solve them.

What's missing is a shared **opinion** on how the team writes tests: when to use a slice test versus a full `@SpringBootTest`, when a Testcontainer is warranted, and how to keep state clean between tests.

Read together with Q3

22% of all engineers (Q3) say *build & test times* are their #1 frustration. For Spring Boot teams (Q6), **only 24%** blame infrastructure (Spring context + Testcontainers). The other **76%** blame strategy, design and discipline.

You can't buy your way out of this with faster hardware.

§ 07 / ANALYSIS

The acceleration whiplash.

AI generates more code — faster. Everything *downstream* of that code generation has to absorb the new volume. Three independent datasets now point to the same place.

90%

ENGINEERS USING AI AT WORK (DORA 2025)

"AI adoption is near-universal". 80% report higher productivity, but 30% have little or no trust in AI-generated code.

−19%

PRODUCTIVITY CHANGE FOR EXPERIENCED DEVS USING AI (METR RCT, 2025)

Developers *believed* AI made them 20% faster. The randomized controlled trial showed they were 19% slower on real tasks.

+441%

MEDIAN TIME-IN-PR-REVIEW CHANGE AMONG 22K AI-USING DEVS (FAROS AI, 2026)

The throughput moved up; the review tax exploded behind it. Incidents-per-PR also rose 242%.

What our 92 engineers are telling us

The DORA report calls AI a *"mirror and a multiplier"*. Our data shows the mirror clearly. Engineers don't blame AI in the abstract — they blame the work AI creates downstream:

- **25% in Q3** point to AI-generated code itself — analyzing it (15.6%) and debugging it (9.4%).
- **11% in Q2** independently surface "AI conversations", "reviewing generated code", and "bigger, more complicated AI MRs" as a top blocker.
- **37% in Q3** point to slow or flaky test pipelines — the safety net that has to verify the new volume.

The thesis of this report

AI didn't create most of these problems. It **exposed** them. Teams that already had fast, reliable tests are amplified by AI. Teams with slow, flaky pipelines and unaligned testing strategies experience what Faros calls the *acceleration whiplash*: more code merged, more bugs to catch, and a review tax that wipes out the speed gain.

§ 08 / WHAT TO DO ABOUT IT

Five moves that pay for themselves.

Each recommendation is tied to a specific number in this report. They are ordered by leverage — how much of the pain a single move removes.

01 Write a one-page testing strategy — and enforce it.

The biggest Spring Boot bottleneck is not flaky tests; it's that **nobody agrees on what to test**. Define which features get unit tests, which get slice tests (@WebMvcTest, @DataJpaTest), which get @SpringBootTest with Testcontainers, and which get end-to-end. Ship the doc. Make it a PR review check.

→ Addresses Q6 #1 (18.95%) + Q6 #4 mock disagreement (15.79%)

02 Cut Spring context startups to one per test suite.

Most Spring Boot suites accidentally rebuild the application context dozens of times because slightly different `@MockBean`, `@TestConfiguration`, or property overrides bust the cache. Audit your suite. One context per slice type. Reuse Testcontainers via singleton pattern. *This single change typically halves CI time on a mid-size codebase.*

→ Addresses Q3 build & test times (21.88%) + Q6 context startup (13.68%)

03 Make the test pipeline the gate for AI-generated PRs.

If AI generates a 30-file PR, a human cannot review every line. **The test suite must.** That means coverage gates, mutation testing for critical paths, and a contract for AI tooling: it must produce *integration* tests, not just unit tests that mock the world. Stop merging AI code that only adds @Mock-heavy unit tests.

→ Addresses Q3 AI-code friction (25.01%) + Q2 AI review burden (10.9%)

04 Quarantine flaky tests within 24 hours.

A flaky test trains your team to ignore red builds. Adopt a hard rule: any test that fails intermittently is **tagged @Disabled with a ticket within one working day**, and the owning team has one sprint to fix or delete it. The signal value of the suite recovers almost immediately.

→ Addresses Q3 flaky tests (15.10%) + Q6 flaky integration (16.84%)

05 Treat test-data management as a first-class concern.

16.84% of Spring Boot teams point here. The fix is unglamorous and reliable: per-test transaction rollback by default, dedicated builders or fixtures for shared data, and Testcontainers reused across the JVM. Avoid `@DirtiesContext`. Avoid shared mutable state between tests. Avoid "just clean the database" in `@AfterEach`.

→ Addresses Q6 test data (16.84%)

§ 09 / THE OFFER

// GUARANTEED DEVELOPER PRODUCTIVITY BOOST

50% Faster Spring Boot Builds *in 30 Days.*

Stop letting AI-generated code turn your team into **exhausted auditors**. Get blazingly fast feedback loops to deploy with **zero fear**.

- 100% RISK-FREE
- 30-DAY GUARANTEE
- MONEY-BACK IF WE MISS

Our blueprint for 50% faster builds

Five steps. No heavy jargon. Real results black-on-white.

01 **The Unvarnished Report**

A ruthless baseline analysis of your project, pipeline, and ways of working to identify hidden productivity killers.

02 **The Build-Time Turbo**

Picking low-hanging fruits directly in the code — Spring context caching, parallelization, Testcontainers optimization — for instant flow.

03 **Pipeline Sanitation**

Eliminating flaky tests, removing duplicate steps, and adding smart caching to build a rock-solid safety net.

04 **The Strategy Rollout**

Handing over a foolproof standard workflow and showing your team the massive time savings black-on-white.

05 **Ultimate Team Enablement**

Hands-on mentoring directly in the code to turn your developers into self-sufficient testing experts.

→ **Want this applied to your pipeline?**

Start with Step 1 — The Unvarnished Report. Book a slot →

Book Step 1 — The Unvarnished Report.

30-minute call. Zero pitch. You leave with at least one concrete optimization you can ship next week — whether we end up working together or not.

Book a slot →

PICK A SLOT

cal.eu/**pragmatech/pragmatech-discovery**

§ 10 / ABOUT THIS REPORT

The whole point of this.

About PragmaTech

PragmaTech helps Java teams build & ship software with confidence — cutting through testing complexity and slow builds for faster feedback and fearless shipping. Founded by **Philip Riecks**, we've spent years inside the engine rooms of startups, scale-ups, and multi-billion-euro enterprises turning slow, flaky test pipelines into fast, trusted safety nets.

This report exists because in 2026, the question changed. It's no longer *"how do we write good tests?"* It's *"how do we validate the 30× more code AI now generates — without our pipeline becoming a maintenance time bomb?"*

Sources cited

Google Cloud / DORA. *2025 State of AI-Assisted Software Development Report.* 90% AI adoption; AI as "mirror and multiplier"; negative relationship with delivery stability.

Becker, Rush, Barnes & Rein. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.* METR, July 2025. Randomized controlled trial: −19% productivity.

Faros AI. *AI Engineering Report 2026.* Telemetry across 22,000 developers: time-in-PR-review up 441%, incidents-per-PR up 242%.

You will be the first to know

You were one of 60 engineers who left an email at the end of this survey. The next edition of the report — with year-over-year comparisons and a deeper cut on AI-generated code review patterns — lands in your inbox first.